FILES AND THE YAML HEADER

kind	suffix	used by	after the header
`document`	—	built into ODT	markup
`fragment`	.f	`|include`	markup
`module`	.m	`|import`	definitions only
`styles`	.s	`styles:` of a project/document	code (Python)
`project`	.p	`project.p.prml`	code (Python)

Kind: kind → suffix → document. UTF-8, LF, NFC. The header is YAML, without expressions.

```
---
kind: document
primula: {version: "1.1", stdlib: "1.1"}
doc: {title: "Договір", language: uk-UA}
params: {vat: true}       # |(vat)
references: [contract]    # project documents for |@
---
```

Keys: `kind primula doc params structure styles references`; in `doc`: `title subject description creator organization language identifier revision status created modified keywords`.

Section name = name in the text and code: `doc:` → `|(doc.title)`. In a project — `doc_defaults`, `ref_documents`; the former `document`/`documents` are error P1120.

WHAT FOLLOWS |

`||`	literal `|`
`(`	expression `|(…)`
letter	long command `|name` (a dot — module)
punctuation, symbol	short form (on the right)
space, end of line, digit	error; also `|$ |[ |]`, `|–` (U+2013) instead of `|-`

LONG FORM OF A COMMAND

```
|name(arg, k=v){body}     inline
|name(arg, k=v): text     line form (first in the line)
|name(arg):               environment
  …
|end name
```

- no spaces between the name, `(` and `{`; arguments are core expressions, may span lines
- a letter right after a command → separator: `|seller|;ом`; a literal `:` — `|seller|::`
- the body `{…}` is counted by brace balance; a single brace — `|{ |}`
- the body of a macro is the parameter `body` (`|(body)`); dynamic call: `|(fn(x))`, `|call(fn, x){…}`

CLOSING ENVIRONMENTS

`|end`	the nearest environment (not if/for/def), unchecked
`|end> |end>> |end.Name |end!`	clause of a level · style · block `|!`, checked
`|end name`	any named one: `|end table`, `|end clause`, `|end style`, `|end admonition`
`|endif |endfor |enddef`	control constructs

SHORT ↔ FULL FORM (SAME RESULT)

short	full
`|*{x} |/{x} |_{x}`	`|.Strong{x} |.Emphasis{x} |.Underline{x}`
`|^{x} |,{x} |-{x}`	`|.Superscript{x} |.Subscript{x} |.StrikeText{x}`
`|`{x} |?{x}`	`|.Code{x} |.Placeholder{x}`; `|end>` = `|end clause`
`|<<< Назва`	`|<(level=3) Назва`
`|> текст`	`|>: текст |end>` — explicit scope
`|!("note") текст`	`|!("note"): … |end!`
`|: a |& b`	`|row: |cell: a |end cell … |end row`
`|if(c){…} |for(x in xs){…}`	`|if(c):…|endif  |for(…):…|endfor`

SHORT FORMS — THE COMPLETE LIST

`|| |{ |} |;`	literals `| { }` · empty separator (end of a command name)
`|%`	comment to the end of the line
`|(expr)`	value of an expression (as text, not markup)
`|\ |~`	line break · no-break space
`|*{} |/{} |_{}`	bold · italic · underlined = `|.Strong{}` `|.Emphasis{}` `|.Underline{}`
`|^{} |,{} |-{}`	superscript · subscript · struck out
`|`{} |"{} |?{}`	monospaced (`Code`) · quotes of the language: uk, ru «…» „…“, en “…” ‘…’ · draft placeholder (`--strict` — error; `placeholders("final")` — plain text)
`|.Name{…} |.(expr){…}`	character style
`|.Name: |.(expr):`	paragraph style, line or environment; closing `|end.Name` `|end style`
`|@("id")`	cross-reference, mode — the second argument
`|link("url"){text}`	external hyperlink; without a body the text = the address; style `structure_styles(link=)`
`|< … |<<<<<< |<(level=N)`	heading of level 1–6; `style="Name"` — paragraph style of the heading
`|> |>> …`	clause, level = number of `>`; closing `|end>` `|end clause`; `style=` — style of the clause
`|- |# |-- |#-`	list item: length — level, last character — type (`-` bullet, `#` number)
`|!("kind")`	warning/note; closing `|end!` `|end admonition`
`|: |& |=(expr)`	row = `|row` · cell = `|cell` · deferred cell formula; `|: |& Б` (with a space) — empty first cell

Every short form takes `(arguments)`; `|> |- |#` have no `level` parameter. Structural ones (`|< |> |- |# |! |:`) — only at the start of a line.

CODE, COMMENTS, VERBATIM TEXT

`|% text`	line comment
`|comment: … |end`	block comment (only the balance is checked)
`|verbatim: … |end`	verbatim text
`|verbatim(end="EOF"): … EOF`	own end marker; compared literally
`|code: … |end`	Python subset; in `.s/.p` — without a wrapper, comments `#`

The indent of the opening line is removed from the whole block; another indent is an error.

PARAGRAPHS AND TEXT

- a blank line — a new paragraph; a line break inside a paragraph — a space
- a line form (clause, list item, table row, heading) ends at the end of the line; several paragraphs — an environment
- `None bool list record` are not allowed in `|(…)` (use `str()`, `join_list`); markup from a string — only `markup(…)`
- fractional numbers and dates — through `num`, `fmt_date`, otherwise a warning

CONDITIONS, LOOPS, MACROS, VARIABLES

`|if(n > 3){…}|elif(c){…}|else{…}`	inline
`|for(x in xs){|(x) }`	inline
`|if(c): … |elif(c): … |else: … |endif`	
`|for(it in items): … |endfor`	
`|def sign(name, role="директор"): …|(body)… |enddef`	macro
`|let(rate=20, export=True, override=False)`	`|export(months=12)`, `|export("sign")`, `@export` over `def`
`|>(when=has("wifi")) …`	`when=` — on any block command and table row

`has("x")` by `features: [x]`. `--review` turns on all variants with marks. `|let` / `|code:` inside a top-level `|if` — error.

HEADINGS AND CLAUSES

form	arguments
`|< … |<<<<<<`	`id number level toc toc_title toc_level style when`; `|<(number=False)` — unnumbered; `level=` or the number of `<`, not both
`|> |>>`	`id continues block style toc toc_title toc_level when`; `|>(continues="id")` — continue the numbering
`|>(…): … |end>`	clause of several paragraphs (explicit scope)

The implicit scope of a clause ends with: a paragraph, the next `|>` of the same/higher level, `|<`, `|page section appendix scope document toc list_of header footer`. Lists, tables, `|!`, figures, blank lines — inside the clause.

LISTS

```
|- bullet            |# number
|-- 2nd level        |#- bullet under a number
|#(id="st", start=3) named; |#(continues="st")
|list("Style")       style of the nearest list
|list("Style"): … |end list   (blank lines inside)
```

Do not mix `-` and `#` on one level and do not skip a level.

STYLES, DEFAULTS, BLOCKS

`|.Name{…} |.Name: …`	character / paragraph; `|.(expr)` — the name is computed
`|.Body(before="l", after="s", indent=2)`	parameters in place: step of the spacing scale or a length
`|.(Indented("Clause",2)):`	derived style (`derive`)
`|set(paragraph=, list=, table=, heading=, clause=, …)`	styles of constructs to the end of the environment; `list=None` — reset
`|block("Name"): … |end block`	style of a group; or `block=` of `|> |! |table |figure |appendix`
`|fmt(color="#C00000", bold=True){…}`	direct formatting (warning)

REFERENCES |@

record	result
`|@("id")`	number «1.2»
`|@("id","label")`	«п. 1.2», «розд. 2», «рис. 3», «табл. 1», «дод. А» (en: cl., sec., fig., tab., app.)
`|@("id","title")`	title / text of the first paragraph of the clause
`|@("id","page")`	page number (a Writer field)
`|@("id","label",case="loc")`	«розділі 2»; cases `nom gen dat acc ins loc`

`id=` (`-` `.` allowed; unique) — of a heading, clause, figure, table, appendix, footnote, section, `|!`; in a compound document `act1:items`.

Another document of the project: a document with `references: [contract]` looks for an ID it lacks in the `ref_documents` of the project (the text — from the registry `*.refs.json`); the reference is text without a hyperlink, `page` — error.

APPENDICES AND NUMBERING SCOPES

```
|appendix(id="a1", title="Специфікація",
  subtitle="до Договору", number=3 | letter="Б",
  page="Landscape", block="…"): … |end appendix
|scope("name", prefix="A"): … |end scope
```

A new page; «Додаток 1» (business) / «ДОДАТОК А» (report); own counters: «Таблиця А.1». The name `document` is reserved. Settings — `appendices(designation=, letters=, title_format="{designation} {title}", toc=)`; an appendix is a heading of the structure (Writer navigator, PDF bookmarks).

INCLUDING FILES

`|include("f.f.prml", part=, ns=, rename={}, level_shift=n, k=v)`	fragment; `path part key` known when the tree is built; `level_shift` 0–5 — shifts headings down; `ns/rename` — against conflicts
`|include(key="k")`	`|part("x"):…|end part`
`|import("lib/b")`	module: `b.sign()` · without prefix `|import("lib/b","a","b")` · alias `alias="x"`; `_name` is not exported
`|document("c.prml", scope=, params={}, page=, level_shift=)`	part of a compound ODT; reference `|@("c:id")`

A path is relative to the file, not above the project root; `.prml` may be omitted. Keys of the map — `file#part` for `|include` only.

TABLE |TABLE(…): … |END TABLE

parameter	value · default
`cols`	list of columns; equal by default (or `cols` of the table style)
`style width align`	style · `"100%"` · `left center right`
`header footer`	header rows (1) · total rows (0)
`repeat_header keep_rows may_break`	all `True`
`frame rules`	frame · `all rows cols none`
`caption id`	numbered caption · target of references
`block when live`	block style · condition · `live=True` — Writer formulas
`data fields titles`	table from a list of records: `|table(data=items, fields=[…], titles=[…])`

Only rows, `|caption:`, `|for|if`, `|code:`, `|include`, comments are allowed inside. A caption with markup: `|caption: … |end caption`. Space before/after a table — `space_before/space_after` of its style (`AdmonitionTable` 3 mm).

COLUMNS COLS=[…]

`"40mm" "30%"`	fixed · share of the table
`"*" "2*"`	the rest, by weight
`"auto"`	by content
`"20mm..60mm" "20mm.." "..60mm"`	within / lower bound only / upper bound only
`col(width, id=, align=, valign=, type=, currency=, digits=, paragraph=, min=, max=)`	`type="decimal|money|datetime"`; `currency="UAH"` → «грн»

ROWS AND CELLS

```
|: Назва |& К-сть |&(span=2, align="right") Разом
|:(keep=True, when=has("wifi")) Умовний рядок |& 1
|:|&(span=3) Разом       arguments of the 1st cell
|: |& Б                  empty first cell
|row:   |cell(rows=2, valign="top"): paragraphs, lists,
  nested tables |end cell |end row     expanded
```

cell	`span=N` horiz. · `rows=N` vert. · `align valign padding background paragraph border border_top border_bottom border_left border_right`
row	`height min_height keep when` + cell properties

Cascade: table → column → row → cell. Cells in a row = columns (counting merges); an empty one — `|&`. Border: `"0.5pt solid #000000"` (`solid dashed dotted double none`).

CELL FORMULAS |=(…)

`this_row(c) cell(r,c)`	value in the current row · cell (from 1; column — number or id)
`column(c, rows=…) body_rows()`	values of a column · rows between header and totals
`sum min max avg count`	aggregates; `None` is skipped

```
|:|&(span=2, align="right") Разом
  |& |=(sum(column("sum", rows=body_rows())))
```

CAPTIONS, CONTENTS, FOOTNOTES, TERMS

```
|toc(max_level=3, title="ЗМІСТ", page_break=True)
|list_of("figure"|"table"|"abbr"|"index", title="…")
|footnote(id="fn"){text}  |endnote{text}  |@("fn")
|abbr("ПЗ","expansion"[, show=False][, text=])
|abbr("ПЗ")      |index("напруга", sub="живлення")
|toc_entry("Text", level=1)    contents line not from a heading
```

Clauses `|>` are not in the contents; `toc=True` includes a clause or a deeper heading; `toc=False`, `toc_title=`, `toc_level=` — on `|<`, `|>`, `|appendix`.

ADMONITIONS |!

```
|!("warning") One line.
|!("danger", hazard="Висока напруга", consequence="…",
   symbol="assets/w012.svg", id=, block=, when=):
  …paragraphs, lists…
|end!
```

Kinds: `danger` DANGER · `warning` WARNING · `caution` CAUTION · `notice` NOTICE · `note` Note · `tip` Tip (words by the document language). `hazard` is required for `danger` / `warning`.

Primula v1.1-draft · cheat sheet

PAGES, BREAKS, HEADERS AND FOOTERS

```
|page("Appendix", restart=1, number_format="i")
|page("Landscape"): … |end page
|newpage      |newcolumn
|section(cols=2, gap="6mm", rule="0.5pt solid",
  balance=True, widths=["60%","40%"], id=):
  … |end section
|header("Portrait", which="all|first|left|right"):
  …markup… |end header    (|footer — the same)
```

Fields: `|field.page |field.pages |field.chapter(level=1) |field.title`. `page_style(…)`: `parent` `size` (`A3 A4 A5 Letter` or `(w,h)`) `orientation` `margin` (top, right, bottom, left) `cols` `col_gap` `col_rule` `header_gap` `header_min_height` `footer_gap` `footer_min_height` `number_format` `restart` `next` `background` `watermark` `mirror`.

PARALLEL TRACKS (§7.7)

```
|parallel(langs=["uk","en"], gap="8mm", rule="0.3pt solid",
  parity="structure|translation|none"):
|segment(id="s") |track("uk"){…} |track("en"){…}
|segment(id="t"):  |track("uk"): …  |track("en", rev=, status=): …
  |end segment     |segment(span="all"): … |end segment
|end parallel      track("note", width="30%", numbered=False,
  optional=True, primary=False, title="…", lang="en")
```

ID — `uk.id`; `|toc(track="en")`; `primula parallel F --stub|--stamp --track en [--segment s --status reviewed]|--check-status reviewed`.

GRAPHICS

```
|img("logo.png", height="8mm")   |img(key="logo")
|img(svg=bar(30))   |figure(chart(d)): …   # SVG from code
|img(svg=load_text("img/знак.svg"))         # SVG from a file
|figure("scheme.svg", width="120mm", id="fig-1"):
  Підпис рисунка |end figure
|frame(rel="page", x="120mm", y="25mm", width="70mm",
  wrap="through", layer="front"): … |end frame
|watermark("ПРОЄКТ", pages=["Portrait"], angle=45,
  color="#C8C8C8", size=None, opacity=50, font=None)
```

place	`inline left center right inside outside`; `valign top middle bottom`; `x y`
rel anchor	`para area page` · `char para page`
wrap layer	`none side both auto through contour` · `front back`
other	`gap width height rotate mirror crop opacity border padding style alt`

PNG, JPEG, SVG; paths relative to the file, inside the project. One of the forms: a path, `key=`, `svg=` or a value `img(…)`. Properties of a frame style are the defaults of the `|frame` arguments.

STANDARD LIBRARY

numbers	`num(x,digits=) percent round_to(x,step) ceil_to floor_to round_money`
money	`money(x,"UAH") money_words money_full(kop="digits|words", cap=, currency_style="short|full") vat(net,rate) with_vat vat_in without_vat split(total,[30,70])`
in words	`words(n, gender="m|f|n", case=) ordinal date_words`
dates	`add_months add_years weeks hours minutes fmt_date(dt,"short|long|contract|{d} {month_gen}") fmt_time fmt_datetime weekday month_name workdays(d,n,calendar=) is_workday`
units	`qty(x,"V",tol=) qty_range(a,b,u) convert(x,"kW","W")`
text	`plural(n,"день","дні","днів",frac=) join_list(xs,last=,sep=) cap translit sort_key nbsp_short`; all text — `typography(nbsp_short=True)`
data	`group_by sum_by distinct`; `|import("std/math")`: `sqrt exp log log10 sin cos tan atan2 pi e floor ceil`
QR	`qr_svg(data, ecc=, size=, circle=) qr_matrix base64url(text, encoding=)`

Language — `lang="uk\|ru\|en"`. A `float` in money functions is an error (use `dec()`). The VAT rate is passed explicitly.

FREQUENT MISTAKES

- a space, digit, end of line after `|`; `|–` (U+2013) instead of `|–`
- `|sellerом` reads as the command `sellerом` — write `|seller|;ом`
- `|end` does not close `|if|for|def` — use `|endif|endfor|enddef`; `|code:` — only with a bare `|end` line
- the number of `<` together with `level=`; a skipped level of a heading or list
- a string instead of a boolean in `when=`; `|let` in a top-level `|if`
- the key `lines` and `primula: {version: "1.0"}` — errors (the line mode is gone)
- `document:`/`documents:` in the header — error P1120 (now `doc`, `doc_defaults`, `ref_documents`)

PROJECT PROJECT.P.PRML

```
---
kind: project
name: "Договори 2026"    language: uk-UA
doc_defaults: {organization: "ТОВ «Постачальник»"}
styles: [styles/business.s.prml]
on_conflict: error    build_date: 2026-10-12
params: {city: "м. Харків"}    keys: {logo: a/l.svg}
ref_documents: {contract: c.prml}
extends: ../../project.p.prml
limits: {time: 20, recursion: 500, size: 10_000_000}
targets:
  - {source: c.prml, output: out/c.odt, pdf: true}
---
for row in load_csv("data/acts.csv"):
    build("act.prml", params=row,
          output=f"out/{row.no}.odt")
```

Priority of parameters: target → document → project. `batch_mode()` — only in the code of a project. Data: `load_csv(path, types={…}, sep=)` `load_yaml` `load_json` `load_ods` `load_xlsx` `load_text`. `extends:` — the project inherits a base one (parameters, keys, `ref_documents`, `doc_defaults`, styles, code; targets are not inherited). `doc_defaults` — default ODT metadata; a document replaces them one by one.

STYLES (MODULE .S.PRML)

YAML groups	`paragraphs characters blocks tables lists frames pages` = functions `paragraph_style character_style block_style table_style list_style frame_style page_style`; inheritance `parent`
paragraph	`font size bold italic color background underline strike language align line_height space_before space_after left right first_line keep_with_next keep_together page_break_before widows orphans`
table	`width align frame rules border padding background header_* footer_* paragraph repeat_header keep_rows may_break space_before space_after cols`
list	`paragraph levels:{1:{type format prefix suffix bullet left hanging start letters}}`

Settings: `structure_profile("business\|report")`, `structure_styles(link=, clause_number=, heading_number=, …)`, `numbering`, `captions`, `appendices`, `ref_labels`, `footnotes`, `quotes`, `abbreviations`, `admonition_words`, `placeholders("mark\|final")`, `typography(…)`, `derive(base, name=, **props)`. Layers: project → `--style` → `styles` of the document → code of the document; `extends: std/styles/business` — base of a module, a layer before it. Sets of the kit `std/styles/…`: `business letter technical report compact` (base `default`).

COMPUTING CORE

- types: `None bool decimal float str list tuple dict record datetime length content function`; an integer is `decimal`; `2.20` → «2.20»; `1.5e3` — `float`; `round` — away from zero
- lengths: `10*mm`, `"10mm"` (`mm cm pt inch`); dates: `date(2026,10,12)`, `dt + 5` — days
- there are: `def while for if`, comprehensions, f-strings, `*args **kw`; **there are no:** `class import from try except raise with global del lambda yield async await := match`
- functions: `len range enumerate zip sorted reversed min max sum any all abs round int float dec str bool list tuple dict record date datetime type getattr hasattr repr fail debug markup build_date`; methods of `str list dict` as in Python
- top level: definitions (lazy, immutable) and actions; limits `time` 20 s · `recursion` 500 · `size` 10^7
- differences from Python in detail — page 4

COMMAND LINE

```
primula build file [-o out.odt] [--pdf [f.pdf]] [--report r.json]
primula check file [--dump-project | --dump-structure --dump-styles]
                   [--overlay FILE=BUFFER]
primula export file --to md|html|docx [-o out] [--pandoc-arg A]
primula commands [--format json] [--functions | --header]
primula console [file] [--json]
```

`--project F` `--no-project` `--strict` `--review` `--build-date YYYY-MM-DD` `--style F` (repeats) `--param NAME=VALUE` (repeats) `--format text|json` `--lang` `--timeout SEC` `--unsafe` `--embed-sources` `--no-embed-report`. Exit code: 0 success, 2 errors. Diagnostics `P0…P9`: CLI/input-output · syntax · names/core · styles · structure/references · tables · pages/graphics · modules/project · ODF · technical documents.

`--dump-*` and `commands` give JSON for editors and Studio; `console` — trying code and library functions (`:help name`, `:functions`, `:vars`, `:reset`, `:context`, `:quit`).

NUMBERS: DECIMAL INSTEAD OF INT AND FLOAT

An integer and a number with a point without an exponent are `decimal`: exact, up to 64 significant digits, keeps the number of decimals. There is no `int` type: `type(5) == "decimal"`. `float` — only a literal with an exponent `1.5e3` or `float(x)`; `decimal` with `float` → `float`.

Expression	Primula	Python 3
`0.1 + 0.2 == 0.3`	`True`	`False`
`2.20 * 3`	`6.60`	`6.6000000000000005`
`7 / 2`	`3.5` decimal	`3.5` float
`1 / 3`	28 significant digits	16–17 digits
`round(2.5)`, `round(-2.5)`	`3`, `-3`	`2`, `-2`
`round(2.675e0, 2)`	`2.68`	`2.67`
`-7 // 2`, `-7 % 2`	`-4`, `1`	the same
`2 ** -1`	`0.5` decimal	`0.5` float
`10 ** 70`	P2309 (>64 digits)	integer
`True + 1`	P2304	`2`
`[1, 2][2.0 - 1]`	`2`	TypeError

One rounding — half away from zero, `float` — by its decimal notation. Money — only `decimal` (`dec("0.20")`); money functions do not take `float` (P2703). Into the text — `num money percent`, not `f"{x:,.2f}"` (the Python format is English).

STRINGS

- Unicode characters, NFC: `len("Київ") == 4`, slices and iteration by characters
- methods — only: `split rsplit splitlines join strip lstrip rstrip replace startswith endswith find rfind count upper lower capitalize title isdigit isalpha isspace zfill format`; others (`center partition encode`) — P2313
- f-strings by the 3.11 syntax: no quotes of the same kind inside `{…}` (P1208) — `f"{d['k']}"`
- `"{0.name}".format(r)` — P2330; a record field — `"{0[name]}"`; `"%s: %d" % (a, n)` is allowed
- `"x" * 2.5`, index `0.5` — P2322 (an integral value is needed)

LISTS, TUPLES, DICTIONARIES

- list and dictionary comprehensions, unpacking `a, *b = xs` — as in Python
- there are no sets: `{1, 2}` — P2101; uniqueness — a dictionary or `not in`
- no operators `| & ^ << >> ~`; merging dictionaries — `{**a, **b}`
- `is` / `is not` — only with `None` (P2103)
- a dictionary key is immutable: `None bool decimal str datetime`, a tuple and a record of them; `float` and a length — P2317; `5` and `5.00` — one key
- `range()`, `reversed()` return a list; `key=` — an ordinary `def`, no `lambda`
- list methods: `append extend insert pop remove index count sort reverse`; dictionary: `get keys values items update pop setdefault`
- `[1] + (2,)` — P2301, like TypeError in Python

RECORDS RECORD

```
r = record(назва="Папір", кількість=3)
r.назва   r["кількість"]   hasattr(r, "назва")
getattr(r, "ціна", 0)      r.replace(кількість=4)
dict(r)   record(**d)
```

Immutable, **not a sequence**: `"x" in r`, `len(r)`, iteration are errors (in a `namedtuple` `in` searches the values!). A missing field — P2314. Records are YAML parameters, project data, `load_csv/yaml/json`; YAML: `15.00` — decimal, `2026-10-12` — datetime. A dictionary stays a dictionary: `d.a` — error.

WHAT IS MISSING — WHAT INSTEAD

`import`, `from`	`|import("std/math")`, modules `.m.prml`
`class`	`record(…)` and functions
`try except raise`	an error stops the build; `fail("…")`, `getattr(…, default)`, `d.get`
`print open with`	`|(x)`, `debug(x)` — into the report; `load_*`
`lambda yield async`	`def`, a function building a list
`global nonlocal del`	a new name
`for … else`, annotations	a flag, `any()`; no types
`x.__class__`	`type(x)` → the string `"decimal"`
clock, `random`	`build_date()` (`--build-date`); no random numbers

DATES AND LENGTHS

Expression	Primula (Python)
`date(2026,10,12) + 30`	`date("2026-11-11")` (TypeError, needs timedelta)
`d2 - d1`	days, `decimal`, may be fractional (timedelta)
`d + hours(36)`, `weeks(2)`	numbers of days
`add_months(date(2026,1,31), 1)`	`2026-02-28`
`d.weekday`	attribute 1–7, Mon = 1 (method 0–6)
`str(dt)`	`2026-10-12T14:30:00` (with `T`)
`float(d)`	days since 30.12.1899
`10*mm + 1*cm`	`length("20mm")` — unit of the left one
`72*pt == inch`, `(210*mm).cm`	`True` (tolerance 0.001 mm), `21`

One type `datetime` without `import`, precision — a second, no time zones. Lengths: `mm cm pt inch` (not `in`); `length * length` — P2301. Into the text — `fmt_date(d, "contract")`.

GLOBAL NAMES: LAZY, IMMUTABLE

```
total = price * qty      # above the definitions — fine
price = 250.00
qty = 4
rate = 20 if vat else 0  # not: if vat: rate = 20
rows = [f(x) for x in xs]  # not: rows.append(…) in a loop
def build(): …           # or a function returning a list
```

Definitions (`x = …`, `a, b = …`, `def`, `|let`, header parameters) are collected from the whole file, computed at the first use and frozen; their order in the text does not matter. A definition nobody reads is not computed — and its error is not seen. Other top-level statements (`for if` calls) are actions: they run after the collection, their names are local.

`x = 1` … `x = 2`	P2203; a header parameter in code — too
top-level `x += 1`	P2105
`a = b + 1; b = a + 1`	P2204 cycle
`for r in xs: rows.append(r)`	P2318 frozen
`for r in xs: total += r`	P2202 + P2501

FUNCTIONS

`def` with defaults, `*args **kw`, `*, b`, recursion, nested functions and closures, `while`; inside a function variables are mutable. No `lambda yield global nonlocal`. A default is frozen: `def f(x, acc=[])` — P2318 on `append`; a global list passed to a function is frozen too. Limits instead of `RecursionError`: depth 500 (P2402), 20 s (P2406), 10^7 elements.

MODULES AND FRAGMENTS

```
|import("lib/ціни")              ціни.з_пдв(x)  |ціни.macro(…)
|import("lib/ціни", "з_пдв")     з_пдв(x)
|import("lib/ціни", alias="ц")   ц.з_пдв(x)
# in a fragment (.f.prml): names are private, exported explicitly
@export                          # a function under its own name
def знижка(сума, р): …
@export(name="g")                # under the name g
def _g(x): …
|export("підпис")  |let(x=1, export=True)  |export(y=2)
|include("f", ns="f")   # exports into f.*; rename={…}
```

Module: all names without `_` are open, no export needed; `module._name` — error. The `|import` of a document is visible in fragments, the `|import` of a fragment — only in it; the path is relative to the file. `@export` is the only decorator: outside a fragment P2610, over a nested function P2106, in a module — P2638.

VALUES IN THE TEXT

There is no printing: only `|(x)`. Inserted are `str`, numbers, dates, `content`; `None bool list dict record` — P2603 (not «None»): `|(x or "—")`, `join_list(xs)`, `str(x)`. An unformatted fraction or date — P2604. `markup("…") + "…"` — P2301: strings first, then `markup()`.